

Eric Ijeoma

+2347037257767 | ericijeoma7767@gmail.com | linkedin.com/in/eric-ijeoma | github.com/ericijeoma

Education & Certificates

- | | |
|---------------------------------------|-----------|
| • AI/ML Engineer Certificate – Micro1 | Completed |
|---------------------------------------|-----------|

Work History / Projects

AI/ML Engineer at Lincorne Technologies, Remote

2024 – Present

- Led a 6 person AI/ML team that built and managed a system where multiple AI helpers (Multi-Agent Orchestration), a smart language brain (LLM), and live health monitoring (DevOps Observability) work together like different departments in a company, each one doing its part, so that real business users get accurate answers to complex questions in under 0.5 seconds.
- Conducted code reviews using GitHub pull requests with CI/CD checks across the team's pipeline and agent contributions, because multiple engineers modifying the same multi-agent codebase without review created a high risk of conflicting logic or untested changes reaching production. This resulted in more consistent code quality and fewer regressions across the team's agent pipeline contributions.
- LangGraph was used to map out and enforce the exact step-by-step workflow each AI agent follows when passing a task to the next agent — because without a defined structure the agents would conflict, repeat work, or lose track of the conversation. This resulted in a fully stateful multi-agent pipeline exposed via REST APIs that serves many users simultaneously with sub-500ms streaming response times in live production.
- RAG, NLP, and SQL were used to query structured financial databases and extract only the most relevant document sections before the LLM generated any answer. This was necessary because LLMs guess from memory when given no source material, which causes frequent errors on specialized financial data. The result was a 40% reduction in hallucinations compared to the base LLM, making the system reliable enough for high-stakes financial decisions.
- MCP (Model Context Protocol) and A2A (Agent-to-Agent) communication were used to give each AI agent a controlled set of tools and a secure private channel to hand tasks to one another — because agents sharing tools without strict rules causes data leaks and conflicting actions that silently break the system — resulting in zero failures or data mix-ups across all concurrent live production sessions.
- Docker, Kubernetes, and AWS were used to package the AI system into portable containers, orchestrate how those containers scale under load, and host the entire infrastructure in the cloud through a DevOps CI/CD pipeline. This was necessary because deploying AI without containers causes environment mismatches between machines and makes production bugs impossible to trace back to their source. The result was consistent, repeatable deployments with full distributed tracing visibility, giving engineers a clear view of exactly what the AI is doing at any moment in production.

- Trained, improved, and launched AI tools (LLM Fine-Tuning, NLP, Python) that automatically handle business tasks like reading emails, searching the web, and updating notes — so the company could stop paying high fees for cloud AI services and run a smarter, faster, cheaper version in-house.
- LoRA Fine-Tuning was used with numpy and PyTorch to retrain a general-purpose LLM on the company's own data through a complete SDLC pipeline covering data preparation, training, evaluation, and deployment — because the base model produced generic answers that did not match the company's specialized domain and racked up high cloud API costs. This resulted in 31% higher accuracy and 60% lower inference cost compared to the original cloud API.
- AWS Lambda, S3, RDS, EventBridge, Github Actions and OAuth2 were used to build a secure AI cost and quality control system that tracked LLM token usage and spending, monitored answer quality, and blocked faulty prompt updates before release. This reduced annual AI token costs by 40% and prevented prompt-related regressions from reaching production.
- Python and System Architecture Design of AI principles were used to plan and build a real-time web intelligence SaaS agent with TypeScript, JavaScript, HTML/CSS dashboard, deployed through a GitHub DevOps pipeline and delivered in Agile sprints — because the business needed a fast, scalable way to collect and process large amounts of web data without getting blocked or rate-limited by websites. This resulted to a live production tool that collected over 10,000 web pages per day, and onboarding its first paying customer within two weeks of launch.